Seminar Guide: Selenide vs. Selenium WebDriver

Part 1 - Selenide Basics
- 1. What is Selenide?
- 2. Maven Project Configuration
- 3. First Commands: open, $, and $$
- 📎 Live Demo: Initialization
- 4. Working with Elements (SelenideElement)
- Scenario: Login on hapifyMe!
- 5. Working with Collections (ElementsCollection)
- 6. Did you know...?
- 7. Real-World Application: "The Phantom of the Missing Element"
- 8. Hands-On Practice: Live Refactoring Exercises
- Reference Solutions for Hands-on Exercises

Part 2 - Smart Waits and Fluent Assertions
- 1. The Selenide Philosophy: An Assertion is a Wait
- 2. Configuring Timeouts globally
- 3. The Three Magic Words: should, shouldBe, shouldHave
- 4. Common Conditions (Condition.*)
- 5. Integrating AssertJ for Complex Validations
- 6. Real-World Scenario: AJAX Post Creation
- 7. Hands-On Practice: Assertions and Waits
- Reference Solutions for Hands-on Exercises

Part 3 - Page Object Model (POM) and Design Patterns
- 1. POM the Selenide Way: Goodbye PageFactory!
- The Selenide.page() Method
- 2. Implementing Pages: Login and Register
- The LoginPage Class
- 3. Fluent Interface (Method Chaining)
- 4. The Builder Pattern: Managing Test Data
- 5. Reusable Components: ElementsContainer
- 6. Live Practice: Build Your Framework
- Reference Solutions for Hands-on Exercises

Part 4 - Refactoring, Configuration, and Advanced Tricks
1. The Migration Cheat Sheet
2. Managing Configuration: The Singleton Pattern
3. Live Demo: The Grand Refactoring
The "Before" (Vanilla Selenium)
The "After" (Refactored with Selenide)
4. Pro-Level Features: The "Magic Tricks"
A. Soft Assertions (Validate Everything, Report at the End)
B. Headless Mode & Resolution for CI/CD
C. File Uploads Made Easy
D. Javascript Execution Fallbacks
🎯 Seminar Wrap-Up & Open Q&A

Part 1 - Selenide Basics

> 📈 **Welcome to the Seminar!**
>
> If you have spent hours building UI automation frameworks manually using core Selenium WebDriver - managing driver lifecycles, writing explicit waits, and dealing with flaky StaleElementReference exceptions - this session is for you. Today, we are switching to "autopilot."
>
> Selenide is a powerful framework built on top of Selenium WebDriver that automatically handles timing issues and makes your test code highly readable, concise, and stable. In this first part of our seminar, we will cover the high-level overview of Selenide and how it eliminates standard boilerplate code.

1. What is Selenide?

Selenide is a Java library used for writing concise, stable, and easy-to-read UI tests. Think of Selenide as a smart wrapper around Selenium WebDriver.

Main advantages over "vanilla" Selenium:

- ✅ **Smart Waiting:** You no longer need to write WebDriverWait for every element. Selenide automatically waits for elements to become visible or clickable.
- ✅ **Driver Management:** You no longer need System.setProperty or complex driver configurations. Selenide opens and closes the browser automatically.
- ✅ **Fluent Syntax:** The code reads almost like a standard English sentence, making it highly maintainable for QA engineers and developers alike.

2. Maven Project Configuration

📖 To use Selenide in your framework, you only need to add a single dependency to your **pom.xml** file. Because Selenide already bundles Selenium and WebDriverManager natively, your configuration file remains extremely clean.

```
<dependencies>
    <!-- Main Selenide dependency -->
    <dependency>
        <groupId>com.codeborne</groupId>
        <artifactId>selenide</artifactId>
        <version>7.0.4</version> <!-- Check for the latest stable version
-->
        <scope>test</scope>
    </dependency>

    <!-- TestNG for test execution -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.9.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

3. First Commands: open, $, and $$

In Selenide, everything is based on **static** imports. The core commands you will use daily are:

- **open(String url)**: Automatically initializes the WebDriver (Chrome by default) and

navigates to the URL.

- **$(selector)**: The equivalent of **driver.findElement**. It returns a **SelenideElement** object.
- **$$(selector)**: The equivalent of **driver.findElements**. It returns an **ElementsCollection**.

📎 Live Demo: Initialization

```
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Selectors.*;
import org.testng.annotations.Test;

public class FirstSelenideTest {

    @Test
    public void openHapifyMe() {
        // Browser configuration (optional, Chrome is default)
        // Configuration.browser = "firefox";
        // Configuration.headless = false;

        // Navigation - The driver is launched automatically here! No
boilerplate!
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");

        // Finding an element (without interacting with it yet)
        // Using a fast CSS selector (String)
        $("#emailId");

        // Using a standard By selector (Selenium style)
        $(By.name("log_email"));
    }
}
```

4. Working with Elements (SelenideElement)

Once we have located an element using $, we can interact with it. Selenide has specific methods that differ slightly from standard Selenium to improve stability.

Selenium Action	Selenide Action	Why it's better

element.sendKeys("text")	**element.setValue("text")**	setValue clears the existing text before typing, preventing concatenated strings.
element.click()	**element.click()**	Selenide automatically waits for the element to become clickable before attempting the click.
element.getText()	**element.text()**	Returns the visible text dynamically.
new Select(elem).selectByValue()	**element.selectOption("val")**	Helper methods for dropdowns are available directly on the element itself.

Scenario: Login on hapifyMe!

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;

public class HapifyLoginTest {
    @Test
    public void loginTest() {
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");

        // Using simple CSS selectors based on the 'name' attribute
        $("input[name='log_email']").setValue("george.datcu@hotmail.com");
        $("input[name='log_password']").setValue("ExamplePassword123");

        // Click the login button
```

```
        $("input[name='login_button']").click();

        // Note: Selenide will close the browser automatically at the end
of the test thread.
    }
}
```

5. Working with Collections (ElementsCollection)

If we use **$$,** we get an **ElementsCollectio**n. **Collections** in Selenide are highly powerful because they **evaluate lazily** and allow for **built-in filtering**.

```
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.CollectionCondition.*;

// $$ returns a collection. We can verify its size immediately.
$$(".status_post").shouldHave(sizeGreaterThan(0));

// Get the text of the first post
String firstPostText = $$(".status_post").first().text();

// Filtering: Find all posts containing the text "Hello"
$$(".status_post").filterBy(text("Hello")).shouldHave(size(1));
```

6. Did you know...?

- 💡 **Automatic Screenshots:** Selenide automatically takes a screenshot and saves the page source HTML if a test fails. You can find them by default in the build/reports/tests folder.
- 💡 **Driver Binaries:** You no longer need to manually download chromedriver.exe or set up local paths. Selenide detects your browser version and downloads the compatible driver in the background.
- 💡 **Method Chaining:** You can chain commands to write extremely compact tests: $("#menu").hover().find(".submenu").click();.

7. Real-World Application: "The Phantom of the Missing Element"

> 📝 In standard Selenium frameworks, you have likely encountered a flaky test that randomly fails with a StaleElementReferenceException - often when a page reloads quickly after a save action.
>
> Teams spend hours adding explicit waits or re-initializing elements.
>
> By refactoring these tests with Selenide, this flakiness disappears. Why? Because Selenide's interaction methods (like setValue() or click()) automatically catch StaleElementReferenceException, re-search the element in the DOM, and retry the action seamlessly. It acts as an auto-healing mechanism for your framework.

8. Hands-On Practice: Live Refactoring Exercises

If you have your IDE open, feel free to code along!

- **Exercise 1: Setup and Navigation.** Create a test that opens the Register page (login_register.php) and verifies (using **System.out.println**) the page title.
- **Exercise 2: Fill out the Registration Form.** Using Selenide selectors ($), fill out the registration form (First Name, Last Name, Email, Password) on the right side of the page, but DO NOT click the Register button. Use the setValue method to see it in action.
- **Exercise 3: Collections.** On the login page, use **$$** to count how many <input> elements exist on the page and print the number to the console.

Reference Solutions for Hands-on Exercises

Solution Exercise 1: Setup and Navigation

```java
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;

public class Exercise1 {
    @Test
    public void checkTitle() {
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");

        String pageTitle = title();
        System.out.println("The page title is: " + pageTitle);

        if(pageTitle.equals("Welcome to hapifyMe!")) {
            System.out.println("Correct title!");
        }
    }
}
```

Solution Exercise 2: Registration Form

```java
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;

public class Exercise2 {
    @Test
    public void fillRegistrationForm() {
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");

        // Click the register toggle button
        $("#signup").click();

        // Fill out the fields
        $("input[name='reg_fname']").setValue("QA");
        $("input[name='reg_lname']").setValue("Engineer");
        $("input[name='reg_email']").setValue("qa.engineer@example.com");
        $("input[name='reg_email2']").setValue("qa.engineer@example.com");
        $("input[name='reg_password']").setValue("SecurePass123");
        $("input[name='reg_password2']").setValue("SecurePass123");

        // Short pause just to see it visually during the demo
        sleep(3000);
```

```
    }
}
```

Solution Exercise 3: Collections

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;

public class Exercise3 {
    @Test
    public void countInputs() {
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");

        int inputCount = $$("input").size();
        System.out.println("Total number of input elements on the page: " +
inputCount);
    }
}
```

Part 2 - Smart Waits and Fluent Assertions

> 📈 **Welcome Back!**
>
> In the first part of our seminar, we explored how to initialize the browser and interact with basic elements. However, as anyone who has worked with core Selenium WebDriver knows, the biggest challenge in UI Automation is timing.
>
> Modern web applications are dynamic; they load content asynchronously via AJAX. In "classic" Selenium, this meant writing verbose WebDriverWait statements every time an element was slow to appear. In Selenide, this concept is completely rewritten to be invisible,

> intuitive, and highly stable.

1. The Selenide Philosophy: An Assertion is a Wait

In standard Selenium, we typically separate the action of waiting from the action of verifying:

- Wait for the element (Wait).
- Get the text.
- Verify the text (Assert).

In Selenide, the verification process inherently includes the wait.

When you write **$("#msg").shouldHave(text("Success"));**, Selenide executes the following sequence behind the scenes:

- It checks if the element contains the text "Success".
- If **YES** -> The test passes immediately.
- If **NO** -> It waits 100ms and checks again (polling).
- It repeats step 3 until the condition is met OR the timeout expires.

This built-in polling mechanism is what makes Selenide tests so resilient against UI flakiness.

2. Configuring Timeouts globally

By default, Selenide waits up to 4 seconds (**4000 ms**) for a condition to be met. This is a solid standard for most applications. However, if your test environment is slower, you can configure this globally.

Global Configuration (Recommended in your BaseTest class):

```
import com.codeborne.selenide.Configuration;
import org.testng.annotations.BeforeSuite;

public class BaseTest {
    @BeforeSuite
    public void setupConfig() {
        // Increase the global timeout to 10 seconds
```

```
        Configuration.timeout = 10000;

        // Set a base URL to avoid repeating it in open() calls
        Configuration.baseUrl = "https://apps.qualiadept.eu/hapifyme";

        // Keep the browser open after the test finishes (useful for
debugging)
        Configuration.holdBrowserOpen = false;
    }
}
```

3. The Three Magic Words: should, shouldBe, shouldHave

Selenide uses three primary methods for assertions. Functionally, they are identical (they are aliases of each other), but grammatically, they allow you to write code that reads like Fluent English.

- **should(Condition)**: General use.
  - $("#btn").should(exist);
- **shouldBe(Condition)**: Typically used for states (visible, hidden, checked).
  - $("#btn").shouldBe(visible);
  - $("#checkbox").shouldBe(checked);
- **shouldHave(Condition)**: Typically used for attributes or content.
  - $("#input").shouldHave(value("Smith"));
  - $(".error").shouldHave(text("Invalid Password"));

4. Common Conditions (Condition.*)

The com.codeborne.selenide.Condition class contains dozens of predefined checks. Here are the most useful ones for daily automation:

Condition	Description	Example
visible / appear	The element is visible on the page.	$("#loading").shouldBe(visible);

hidden / disappear	The element is not visible (or does not exist in the DOM).	$("#loading").should(disappear);
text("string")	Contains the given text (case-insensitive, partial match).	$("h1").shouldHave(text("Welcome"));
exactText("string")	The text matches exactly.	$("#title").shouldHave(exactText("HapifyMe"));
cssClass("class")	The element has the specified CSS class.	$("#btn").shouldHave(cssClass("btn-danger"));
attribute("name", "val")	Verifies the value of an HTML attribute.	$("img").shouldHave(attribute("alt", "Logo"));

5. Integrating AssertJ for Complex Validations

Sometimes, Selenide's standard UI checks are not enough. For instance, you might need to verify that a list of strings is sorted alphabetically, or validate a complex date format.

This is where **AssertJ** comes in - a powerful assertion library popular in the Java ecosystem.

When to use Selenide vs. AssertJ:

- **Selenide Assertions (should...)**: Use when verifying UI state (visibility, simple text) and you *need* the smart wait mechanism.
- **AssertJ (assertThat(...))**: Use when you extract data from the page and need to perform complex logical validations on it. **Warning: AssertJ does NOT wait!**

Example:

```
import static org.assertj.core.api.Assertions.assertThat;
import static com.codeborne.selenide.Selenide.*;

// 1. Extract the text (Selenide waits for the element here)
```

```
String profileName = $(".profile_name").text();

// 2. Validate with AssertJ (No wait, pure logic validation)
assertThat(profileName)
    .isNotNull()
    .startsWith("George")
    .endsWith("Datcu")
    .hasSizeGreaterThan(5);
```

6. Real-World Scenario: AJAX Post Creation

Let's look at the classic scenario where Selenium tests often become flaky: submitting a post via AJAX and verifying it appears in the feed without a page refresh.

Scenario: We will log in, submit a new post, and verify it appears.

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Condition.*;

public class AjaxPostTest {

    @Test
    public void createDynamicPostTest() {
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");

        // Quick Login
        $("input[name='log_email']").setValue("george.datcu@hotmail.com");
        $("input[name='log_password']").setValue("ExamplePassword123");
        $("input[name='login_button']").click();

        // Verify we reached the feed
        $(".main_column").shouldBe(visible);

        String uniquePostText = "Hello Selenide " +
System.currentTimeMillis();

        // Write the post
        $("textarea[name='post_text']").setValue(uniquePostText);

        // Click Post (AJAX trigger)
```

```
        $("#post_button").click();

        // THE MAGIC:
        // Selenide will automatically wait until a post containing our
text appears in the feed.
        // No Thread.sleep(), no explicit WebDriverWait needed!

        // Check the entire posts area for our text
        $(".posts_area").shouldHave(text(uniquePostText));

        // Alternatively, find the specific post in the collection and
assert it is visible
        $$(".status_post").findBy(text(uniquePostText)).shouldBe(visible);
    }
}
```

7. Hands-On Practice: Assertions and Waits

If you are following along in your IDE, let's try these out!

- **Exercise 1: Profile Validation.** Log into the application. Navigate to the logged-in user's profile page (click on the name/picture in the top left). Use a shouldHave assertion to verify that the name displayed under the profile picture is correct.
- **Exercise 2: Negative Validation.** On the Settings page (settings.php), attempt to find an element that should NOT exist (e.g., an "Admin Panel" button). Verify that it is not present using shouldNot(exist) or shouldNotBe(visible).
- **Exercise 3: AssertJ Integration.** Navigate to messages.php. Extract the name of the last conversation (the person's name) into a String. Use AssertJ to verify that the name is not null, not empty, and does not contain numbers.

Reference Solutions for Hands-on Exercises

Solution Exercise 1: Profile Validation

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Condition.*;

public class Exercise1_Profile {
```

```
    @Test
    public void profileValidation() {
        // Assuming login steps are completed here...

        // Click the profile link in the navigation bar
        $(".nav-item a[href*='profile.php']").click();

        // Verify the profile name element contains the expected text
        $(".profile_left_col_info").shouldHave(text("George-cristian
Datcu"));
    }
}
```

Solution Exercise 2: Negative Validation

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Condition.*;

public class Exercise2_Negative {
    @Test
    public void negativeValidation() {
        open("https://apps.qualiadept.eu/hapifyme/settings.php");

        // Sanity check: verify a known element is visible
        $("#close_account").shouldBe(visible);

        // Verify a fictional element does not exist
        $("#admin_panel_super_secret").shouldNot(exist);
    }
}
```

Solution Exercise 3: AssertJ

*(**Note**: Requires the **assertj-core** dependency in your **pom.xml**)*

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;
import static org.assertj.core.api.Assertions.assertThat;

public class Exercise3_AssertJ {
```

```
    @Test
    public void messageValidation() {
        open("https://apps.qualiadept.eu/hapifyme/messages.php");

        // Get the first conversation from the list
        String lastPersonName = $$(".loaded_conversations
.user_found_messages")
                                    .first()
                                    .text();

        System.out.println("Name found: " + lastPersonName);

        // AssertJ Validations
        assertThat(lastPersonName)
            .isNotNull()
            .isNotEmpty()
            .doesNotContainPattern("\\d"); // Ensure it contains no digits
    }
}
```

Part 3 - Page Object Model (POM) and Design Patterns

> 📈 **Welcome Back!**
>
> Now that you have mastered elements ($, $$) and smart assertions (shouldHave), it is time to organize our code. If you have ever written tests that degraded into "spaghetti code" full of scattered locators, this chapter provides the solution.

> In Selenide, the Page Object Model (POM) is incredibly simple. Elements are "lazy" - they are not searched for in the browser when defined, but only when you interact with them. This means we can eliminate complex constructors and say goodbye to PageFactory.initElements.

1. POM the Selenide Way: Goodbye PageFactory!

In classic Selenium, initializing elements required boilerplate code:

```
// Classic Selenium (The Old Way)
public LoginPage(WebDriver driver) {
    this.driver = driver;
    PageFactory.initElements(driver, this); // Mandatory boilerplate!
}

@FindBy(id = "user")
WebElement username;
```

In Selenide, elements are defined directly and are ready to use immediately because they are lazy-evaluated:

```
// Selenide Style (The Modern Way)
private final SelenideElement username = $("#user"); // That is it! No
special constructor.
```

The Selenide.page() Method

While defining locators manually using $ is highly recommended for its clarity, if you prefer or need to use @FindBy annotations (e.g., when migrating old code), Selenide handles it gracefully using Selenide.page().

```
import static com.codeborne.selenide.Selenide.page;

public class LoginPage {
```

```
    @FindBy(id = "email")
    public SelenideElement email; // Notice we use SelenideElement, NOT
WebElement
}

// Usage in test:
LoginPage login = page(LoginPage.class);
```

***Note**: **SelenideElement** avoids the **StaleElementReferenceException** trap entirely, unlike Selenium's WebElement.*

2. Implementing Pages: Login and Register

Let's create the structure for the hapifyMe application (https://apps.qualiadept.eu/hapifyme/login_register.php). We will use the recommended manual locator approach.

The LoginPage Class

We define **locators** as **private SelenideElements** and expose **public action methods**.

```
package com.hapifyme.pages;

import com.codeborne.selenide.SelenideElement;
import static com.codeborne.selenide.Selenide.*;

public class LoginPage {

    // 1. Element Definition (Lazy Loading)
    private final SelenideElement emailInput =
$("input[name='log_email']");
    private final SelenideElement passwordInput =
$("input[name='log_password']");
    private final SelenideElement loginButton =
$("input[name='login_button']");
    private final SelenideElement registerLink = $(".signup");

    // 2. Action Methods
    public void openPage() {
```

```
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");
    }

    public void login(String email, String password) {
        emailInput.setValue(email);
        passwordInput.setValue(password);
        loginButton.click();
    }
}
```

3. Fluent Interface (Method Chaining)

To make our tests read like a sentence, we can return the page object (or the *next* logical page) from our methods.

Refactoring login to return a HomePage:

```
// Inside LoginPage.java
public HomePage login(String email, String password) {
    emailInput.setValue(email);
    passwordInput.setValue(password);
    loginButton.click();

    return new HomePage(); // Returns the next logical page
}

// Inside HomePage.java
public class HomePage {
    public final SelenideElement userProfile = $(".user_details");

    public HomePage checkUserIsLoggedIn() {
        userProfile.shouldBe(visible); // Built-in assertion
        return this;
    }
}
```

Usage in Test:

```
new LoginPage()
    .login("user@test.com", "pass")
    .checkUserIsLoggedIn();
```

4. The Builder Pattern: Managing Test Data

When dealing with complex forms, like the Registration form on hapifyMe (First Name, Last Name, Email, Pass, etc.), method signatures become ugly:

register("George", "Datcu", "email...", "pass...") - ***Too many parameters!***

We use the Builder pattern to create a clean User object. In modern Java, we can leverage Records or tools like Lombok, but here is a clear manual implementation.

1. The Model Class (POJO):

```
package com.hapifyme.models;

public class User {
    private final String firstName;
    private final String lastName;
    private final String email;
    private final String password;

    private User(Builder builder) {
        this.firstName = builder.firstName;
        this.lastName = builder.lastName;
        this.email = builder.email;
        this.password = builder.password;
    }

    // Getters...
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
    public String getEmail() { return email; }
    public String getPassword() { return password; }

    // Static Builder Class
    public static class Builder {
        private String firstName;
        private String lastName;
        private String email;
        private String password;
```

```
        public Builder setFirstName(String firstName) { this.firstName =
firstName; return this; }
        public Builder setLastName(String lastName) { this.lastName =
lastName; return this; }
        public Builder setEmail(String email) { this.email = email; return
this; }
        public Builder setPassword(String password) { this.password =
password; return this; }

        public User build() {
            return new User(this);
        }
    }
}
```

2. Usage in the Page Object (RegisterPage):

```
public void registerUser(User user) {
    $("input[name='reg_fname']").setValue(user.getFirstName());
    $("input[name='reg_lname']").setValue(user.getLastName());
    $("input[name='reg_email']").setValue(user.getEmail());
    $("input[name='reg_email2']").setValue(user.getEmail());
    $("input[name='reg_password']").setValue(user.getPassword());
    $("input[name='reg_password2']").setValue(user.getPassword());

    $("input[name='register_button']").click();
}
```

5. Reusable Components: ElementsContainer

Sometimes a page contains a repeating widget (like a user post or a search result block). Selenide provides ElementsContainer to model these discrete chunks.

***Note**: As of newer Selenide versions, you no longer need to explicitly extend ElementsContainer. You can simply define standard classes.*

Example: Modeling a Search Result Block

```
public class SearchResultBlock {
    // These locators are scoped! They will search *inside* the root
element provided during initialization
```

```
    @FindBy(css = ".result-title")
    public SelenideElement title;

    @FindBy(css = ".result-description")
    public SelenideElement description;
}

public class SearchPage {
    // Initialize a collection of component blocks
    @FindBy(css = ".search-result-item")
    public List<SearchResultBlock> results;
}
```

6. Live Practice: Build Your Framework

Let's apply these patterns in our IDEs!

- **Exercise 1: Refactor Login.** Create the LoginPage class containing the locators and methods for logging in (as shown in section 2). Rewrite the test from Chapter 1 using this new Page Object.
- **Exercise 2: Profile Page Object.** Create a ProfilePage class (for profile.php). Add methods to:
  - Get the full name of the user (the text under the profile picture).
  - Post a message on your own wall.
- **Exercise 3: Post Builder.** Create a POJO named PostContent using the Builder pattern that contains: text and an optional youtubeUrl. Use this object as a parameter in your createProfilePost(PostContent content) method.

Reference Solutions for Hands-on Exercises

Solution Exercise 1: Refactor Login

```
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Condition.*;

public class LoginTest {
    @Test
    public void loginRefactored() {
```

```
        LoginPage loginPage = new LoginPage();
        loginPage.openPage();
        loginPage.login("george.datcu@hotmail.com", "ExamplePassword123");

        // Inline verification
        $(".main_column").shouldBe(visible);
    }
}
```

Solution Exercise 2: ProfilePage

```
package com.hapifyme.pages;

import com.codeborne.selenide.SelenideElement;
import static com.codeborne.selenide.Selenide.*;

public class ProfilePage {
    // Locators
    private final SelenideElement profileName =
$(".profile_left_col_info");
    private final SelenideElement postTextarea =
$("textarea[name='post_body']");
    private final SelenideElement submitPostBtn =
$("input[name='post_button']");

    // Methods
    public String getProfileName() {
        return profileName.text();
    }

    public void createProfilePost(String message) {
        // Switch to the posts tab first
        $("#posts_tab").click();

        postTextarea.setValue(message);
        submitPostBtn.click();
    }
}
```

Part 4 - Refactoring, Configuration, and Advanced Tricks

> 📈 **The Final Stretch!**
>
> You now have the building blocks: smart locators, built-in waits, and a clean Page Object Model. In this final part of the seminar, we will focus on the real-world application of Selenide: migrating existing, flaky Selenium codebases, setting up rock-solid framework configurations, and utilizing advanced features for CI/CD pipelines.

1. The Migration Cheat Sheet

If you are **tasked** with **migrating** an **old Selenium** framework to **Selenide**, keep this translation dictionary handy. It highlights how much boilerplate you are about to delete.

Selenium Concept (The Old Way)	**Selenide Equivalent (The Modern Way)**	**Note**
WebDriver driver = new ChromeDriver();	**open("url");**	Driver lifecycle is fully automated.
driver.get("url");	**open("url");**	Concise and intuitive.
driver.findElement(By.id("id"));	**$("#id");**	Returns a SelenideElement.
driver.findElements(By.css("..."));	**$$("...");**	Returns an ElementsCollection (Lazy loading).
element.sendKeys("text");	**element.setValue("text");**	setValue safely clears the field before typing!
wait.until(ExpectedConditions...);	**element.shouldBe(visible);**	Wait is implicitly baked into the assertion.

driver.quit();	***(Nothing)***	Selenide automatically closes the browser thread.
new Select(elem).selectByValue("v");	**element.selectOptionByValue("v");**	Select methods are available directly on the element.
driver.switchTo().alert().accept();	**confirm();**	Built-in static methods for alert handling.

2. Managing Configuration: The Singleton Pattern

While Selenide manages the driver automatically, you still need a single source of truth for your test data (Base URLs, API keys, default credentials). The **Singleton design pattern** is perfect for a **ConfigLoader** that reads from a **config.properties** file once per test run.

A Simple Configuration Manager:

```
package com.hapifyme.config;

public class AppConfig {
    // 1. The single private instance
    private static AppConfig instance;

    // Test Data (In a real framework, read this from a .properties file or
System Env)
    private final String baseUrl = "https://apps.qualiadept.eu/hapifyme";
    private final String validEmail = "george.datcu@hotmail.com";
    private final String validPassword = "ExamplePassword123";

    // 2. Private constructor prevents instantiation
    private AppConfig() {}

    // 3. Global access point (Lazy Initialization)
    public static AppConfig getInstance() {
        if (instance == null) {
            instance = new AppConfig();
        }
```

```
        return instance;
    }

    // Getters
    public String getBaseUrl() { return baseUrl; }
    public String getValidEmail() { return validEmail; }
    public String getValidPassword() { return validPassword; }
}
```

Usage in your BaseTest:

```
Configuration.baseUrl = AppConfig.getInstance().getBaseUrl();
```

3. Live Demo: The Grand Refactoring

Let's look at the core promise of this seminar: taking a clunky Selenium test and turning it into Selenide.

We will refactor the Registration flow.

The "Before" (Vanilla Selenium)

This code is verbose, requires manual wait handling, and is highly prone to StaleElementReferenceException if the page shifts.

```
// DO NOT USE THIS STYLE ANYMORE
@Test
public void oldRegisterTest() {
    WebDriver driver = new ChromeDriver();
    driver.get("https://apps.qualiadept.eu/hapifyme/login_register.php");

    WebElement firstName = driver.findElement(By.name("reg_fname"));
    firstName.sendKeys("Old");

    // ... filling out other fields ...

    // Explicit Wait Boilerplate
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    WebElement btn =
wait.until(ExpectedConditions.elementToBeClickable(By.name("register_button
```

```
")));
    btn.click();

    // Assertion
    WebElement successMsg = driver.findElement(By.id("second"));
    Assert.assertTrue(successMsg.isDisplayed());

    driver.quit();
}
```

The "After" (Refactored with Selenide)

Clean, readable, and incredibly stable.

```
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Condition.*;
import org.testng.annotations.Test;

public class RefactoredRegisterTest {

    @Test
    public void newRegisterTest() {
        // 1. Setup & Navigation
        open("https://apps.qualiadept.eu/hapifyme/login_register.php");
        $("#signup").click(); // Toggle the form

        // 2. Direct, clean interactions (Auto-clears and types)
        $("input[name='reg_fname']").setValue("New");
        $("input[name='reg_lname']").setValue("User");
        $("input[name='reg_email']").setValue("new@test.com");
        $("input[name='reg_email2']").setValue("new@test.com");
        $("input[name='reg_password']").setValue("Pass123");
        $("input[name='reg_password2']").setValue("Pass123");

        // 3. Smart click (Waits for clickability automatically)
        $("input[name='register_button']").click();

        // 4. Fluent assertion (Waits up to 4s for the element to appear)
        $("#second").shouldBe(visible);
    }
}
```

4. Pro-Level Features: The "Magic Tricks"

To elevate your framework from good to great, Selenide offers several advanced utilities built right in.

A. Soft Assertions (Validate Everything, Report at the End)

The Problem: If you are verifying an entire Account Settings form (First Name, Last Name, Email), standard "Hard Assertions" will fail the test immediately if the First Name is wrong. You will never know if the Email was right or wrong until you fix the first issue and rerun the test.

The Solution: Selenide's SoftAsserts executes *all* validations and collects the failures, failing the test only at the very end with a comprehensive list of errors.

Implementation (using TestNG):

```
import com.codeborne.selenide.Configuration;
import com.codeborne.selenide.AssertionMode;
import com.codeborne.selenide.testng.SoftAsserts;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;
import static com.codeborne.selenide.Selenide.*;
import static com.codeborne.selenide.Condition.*;

// 1. Add the Selenide Listener!
@Listeners({ SoftAsserts.class })
public class SettingsSoftAssertTest {

    @Test
    public void checkSettingsForm() {
        // 2. Enable SOFT mode for this test
        Configuration.assertionMode = AssertionMode.SOFT;

        open("https://apps.qualiadept.eu/hapifyme/settings.php");

        // Let's assume we expect "George-cristian" but we intentionally
fail the test:
        $("input[name='first_name']").shouldHave(value("Batman")); // Will
```

```
fail, but test continues!

        $("input[name='last_name']").shouldHave(value("Datcu"));   // Will
pass

        // The listener will collect the "Batman" error and report it at
the end of the method.
    }
}
```

B. Headless Mode & Resolution for CI/CD

When you run your tests on Jenkins, GitLab CI, or GitHub Actions, there is no physical monitor. You must run the browser in memory ("Headless").

The Trap: Headless browsers often start with a small resolution (like 800x600). This triggers the mobile layout of responsive websites (hiding menus behind hamburger icons), causing tests to fail!

The Fix: Always set the browser size explicitly when going headless.

```
@BeforeClass
public void setupHeadless() {
    Configuration.browser = "chrome";
    Configuration.headless = true;       // Run without a GUI
    Configuration.browserSize = "1920x1080"; // CRITICAL: Force desktop
resolution
}
```

C. File Uploads Made Easy

Testing file uploads (like updating a profile picture on upload.php) in raw Selenium is notoriously difficult, especially if the <input type="file"> is hidden by CSS.

Selenide bypasses the OS dialogue entirely and injects the file directly from your project's src/test/resources folder.

```
@Test
public void uploadProfilePicture() {
```

```
    open("https://apps.qualiadept.eu/hapifyme/upload.php");

    // Finds the input (even if hidden) and uploads the file from the
classpath
    File file = $("input[type='file']").uploadFromClasspath("avatar.jpg");

    // Assert upload success...
}
```

D. Javascript Execution Fallbacks

Sometimes, an element is technically in the DOM but hidden behind a sticky header or an invisible overlay. Standard .click() might throw an ElementClickInterceptedException.

Instead of writing verbose JavascriptExecutor boilerplate, Selenide offers a clean fallback:

```
// Execute a click directly via the browser's JS engine
$("#trickyButton").click(ClickOptions.usingJavaScript());
// Or execute arbitrary JS for things like infinite scrolling
executeJavaScript("window.scrollTo(0, document.body.scrollHeight);");
```

🎯 Seminar Wrap-Up & Open Q&A

Thank you for joining this session!

You now have the tools to drastically reduce the size of your testing codebase, eliminate synchronization flakiness, and build an easily maintainable UI automation framework using Selenide.

Key Takeaways:

- **Trust the implicit wait:** Stop writing Thread.sleep() or WebDriverWait. Let .shouldBe() do the heavy lifting.
- **Keep it clean:** Use the Page Object Model with Selenide's lazy-loaded elements.
- **Use advanced features:** Implement Soft Assertions for complex forms and configure explicit resolutions for Headless CI/CD runs.